

# Project 1 - Pseudo Shell

Oliver Boorstein  
University of Oregon

April 21, 2026

# 1 Introduction

This project implements a pseudo shell in C. This shell is replicating the usual Bash or Zsh shell using syscalls and string processing. Overall, the project is fairly simple. We also replicate shell scripts using file mode by running the binary with a file as an argument.

## 2 Background

I had nearly the entire understanding to complete this project before starting. I began this project before we had the lecture on syscalls, but it was fairly simple to figure out their use from online examples. It's nice how the syscalls handle the majority of cases for you.

## 3 Implementation

The design flows through `main.c`. This begins with checking whether we'll be in file mode or shell mode. This is done by checking for a filename argument and validating that argument, mostly through `argc` and `argv[1]`. After that, both modes mostly use the same execution path, which made the project simpler. The only real difference is where the input line comes from.

After this begins the main loop. This loop prompts for input, reads a line (from either `stdin` or a file) and tokenizes it into commands and arguments. This also means that shell scripts work almost exactly like typed commands, since each line is still going through the same parser and command dispatch code.

The command and argument tokenization was one of the few difficult parts of this project. Tokenization uses a dynamic string array which grows as the number of commands or arguments does, using calls like `realloc` when more space is needed. These strings are trimmed to remove white space and `;` to clean up for the main loop.

From there, the loop uses a `if else` chain to determine which command to run. Before running a selected command, the args are checked to avoid errors. This catches cases like `cp` with only one path, `cd` with too many arguments, or `cat` without a filename before reaching the syscall layer.

Command implementations, for many of them, are solved entirely by syscalls. Of particular interest, are `copyFile` and `displayFile`.

`copyFile` is the implementation for `cp`. It opens the source file with `open(sourcePath, O_RDONLY)`, checks it with `stat`, and rejects directories with `S_ISDIR`. If the destination is already a directory, it appends the original filename using `basename`. From there it opens the destination with `O_CREAT | O_WRONLY | O_TRUNC`, so an existing output file gets replaced. The actual copying uses a `char buf[4096]` stack buffer and loops over `read` and `write`, including handling the case where `write` writes fewer bytes than were read.

`displayFile` is the implementation for `cat`. It opens the file with `open(filename, O_RDONLY)`, checks it with `stat(filename, &st)`, and rejects directories before reading. Since the command only needs to print one file, it allocates a buffer with `malloc(st.st_size + 1)`, reads the whole file into memory, and adds `'\0'` before writing it back out to the shell. It then adds a newline and calls `free(buf)` and `close(fd)`, cleaning up the memory and file descriptor.

## **4 Performance Results and Discussion**

Performance is perfectly usable for all functions of this program. No known issues with any functions. Even for reading large text files, the shell is very fast.

## **5 Conclusion**

Overall, a fairly simple project. Much of the implementation is handled by syscalls, meaning most of the work is string manipulation. This lead to some memory debugging, but that was easy to figure out as well. There was also some weird semantics with preventing copying or deleting folders versus files. Stat was helpful here.